SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninfor
solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

Duration : 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Analytical Problem Solving

Code of Conduct:

I SHARWIN(192521207) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Sharwin

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

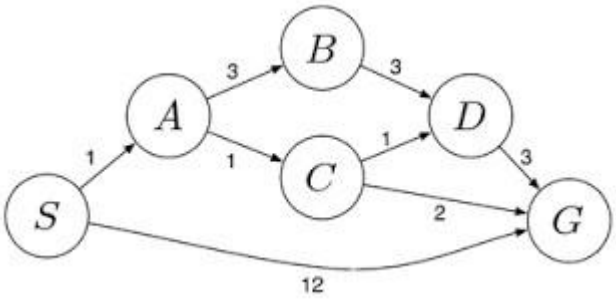
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent? ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Sure. Below is a **more detailed, exam-ready version** of all 5 answers. I've added **definitions, problem formulation, detailed steps, examples, algorithms, advantages, and conclusions** so you can write longer answers in your record/exam.

1. Water Jug Problem – 4 Gallon and 3 Gallon Jugs

Problem Statement

We are given two water jugs:

- One **4-gallon jug**
- One **3-gallon jug**
- Neither jug has any measuring marks.
- There is an unlimited water pump.
- Water can be poured from one jug into another.
- Water can be emptied onto the ground.
- We have to obtain **exactly 2 gallons of water in the 4-gallon jug**. This is a classical **Artificial Intelligence state-space search problem**.

Initial State

Initially both jugs are empty.

\[(0,0) \] Here:

- First value = amount of water in 4-gallon jug
- Second value = amount of water in 3-gallon jug Therefore:

Initial State = (0,0)

Goal State

We need exactly 2 gallons in the 4-gallon jug.

Therefore:

\[\boxed{(2,0)} \]

The second jug can also contain some water, so any state of the form:

\[(2,y) \] can be considered a goal if the requirement is only that the 4-gallon jug contains 2 gallons.

Possible Operations

There are six important operations:

1. Fill the 4-gallon jug completely.
2. Fill the 3-gallon jug completely.
3. Empty the 4-gallon jug.
4. Empty the 3-gallon jug.
5. Pour water from 4-gallon jug into 3-gallon jug.
6. Pour water from 3-gallon jug into 4-gallon jug.

Step-by-Step Solution

Step	State (4G, 3G)	Operation
1	(0,0)	Initial state
2	(0,3)	Fill 3G jug
3	(3,0)	Pour 3G into 4G
4	(3,3)	Fill 3G jug
5	(4,2)	Pour 3G into 4G until 4G is full
6	(0,2)	Empty 4G jug
7	(2,0)	Pour 3G into 4G

Therefore, the final state is:

$$\boxed{(2,0)}$$

So, the **4-gallon jug contains exactly 2 gallons**.

Explanation of Important Step At state: $(3,3)$ the 4-gallon jug already has 3 gallons.

There is only **1 gallon of free space** in the 4-gallon jug.

When we pour from the 3-gallon jug:

- 1 gallon enters the 4-gallon jug.
- 2 gallons remain in the 3-gallon jug. Therefore:

$$(3,3)\rightarrow(4,2)$$

Now empty the 4-gallon jug:

$$(4,2)\rightarrow(0,2)$$

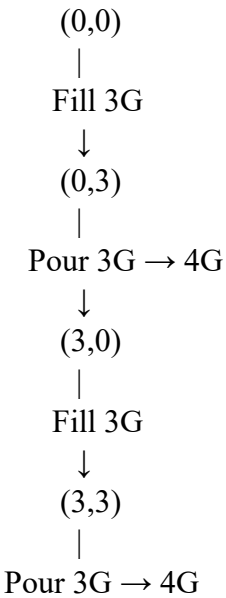
Finally, pour the remaining 2 gallons into the empty 4-gallon jug:

$$(0,2)\rightarrow(2,0)$$

Final Answer

$$\boxed{\text{Exactly 2 gallons are obtained in the 4-gallon jug.}}$$

State Space Representation



↓
(4,2)
|
Empty 4G
↓
(0,2)
|
Pour 3G → 4G
↓
(2,0)
GOAL

Conclusion

The Water Jug problem demonstrates how an AI system can represent a problem as **states and actions** and search for a sequence of actions that reaches the desired goal state.

2. Mars Rover – Intelligent Agent

Introduction

A Mars Rover is an autonomous robotic system designed to explore the surface of Mars. It can:

- Observe the environment
- Move across the surface
- Detect rocks and soil
- Collect samples
- Analyze samples
- Take photographs
- Detect obstacles
- Communicate information to Earth
- Make decisions based on available information

The rover can be considered an **intelligent agent** because it receives information from the environment through sensors and performs actions using actuators.

i. Percepts of the Mars Rover Agent

A **percept** is information received by an agent through its sensors.
The Mars Rover uses different sensors and instruments.

Major Percepts

Sensor/Instrument	Percept
Camera	Images of rocks and terrain
Temperature sensor	Temperature
Spectrometer	Chemical composition
Navigation camera	Position and obstacles
Soil sensor	Soil properties
Radiation sensor	Radiation level
Distance sensor	Distance from objects
Power system	Battery level

Communication system Communication status

Examples

The rover may perceive:

- A large rock in front of it.
- A slope on the surface.
- Low battery level.
- Interesting soil composition.
- A possible sample collection location.
- An obstacle blocking its path.

ii. Characterization of Operating Environment

The Mars Rover environment can be described using the standard AI environment properties.

Property	**Mars Rover**
Observability	Partially observable
Number of agents	Mainly single agent
Deterministic	Uncertain / partially stochastic
Episodic/Sequential	Sequential
Static/Dynamic	Dynamic
Discrete/Continuous	Mostly continuous
Known/Unknown	Partially unknown

1. Partially Observable

The rover cannot observe the entire Martian environment at once.
For example, it may not know what is behind a rock.

2. Sequential

Current decisions affect future decisions.
For example:
Move → Find sample → Collect sample → Analyze sample → Send data

3. Dynamic

The environment can change because of:

- Dust
- Temperature
- Lighting
- Terrain conditions
- Equipment conditions

4. Continuous

The rover deals with continuous values such as:

- Distance
- Temperature • Speed
- Battery level
- Position

5. Partially Unknown

Scientists do not know everything about Mars in advance. The rover must discover new information during exploration.

iii. Actions of Mars Rover

The rover can perform many actions.

Movement Actions

1. Move forward
2. Move backward
3. Turn left
4. Turn right
5. Stop
6. Change speed

Scientific Actions

7. Take photographs
8. Collect soil
9. Collect rock samples
10. Analyze samples
11. Measure chemical composition
12. Measure temperature

Communication Actions

13. Store data
14. Send data to Earth
15. Receive instructions from Earth

Safety Actions

16. Avoid obstacles
17. Stop when dangerous conditions are detected
18. Manage battery power
19. Select safe routes

iv. Performance Evaluation

The performance of the Mars Rover can be evaluated using a **performance measure**. A good rover should complete its mission safely while collecting maximum useful information.

Performance Measures

1. Number of useful samples collected
2. Quality of scientific data
3. Area explored
4. Accuracy of navigation
5. Energy consumption
6. Communication success
7. Number of obstacles avoided
8. Mission completion
9. Rover safety
10. Time required

Example

Suppose two possible routes exist:

- Route A: 5 km, low scientific value
- Route B: 4 km, contains useful samples

The agent can select Route B if it provides greater scientific utility while remaining safe.

v. **Suitable Agent Architecture** A suitable architecture is a:
\[\boxed{\text{Model-Based Utility-Based Agent}} \] A more advanced rover can also use **learning capabilities**.

Why Model-Based?
The environment is partially observable.
Therefore, the rover needs an **internal model** of the environment.
It can remember:

- Previously explored locations
- Obstacles
- Sample locations
- Terrain conditions
- Battery status
- Safe routes

Why Utility-Based?
There may be multiple possible actions.
For example:

- Collect a sample
- Continue exploring
- Return to a safe location
- Recharge
- Send data

The rover can compare these actions based on:

- Scientific value
- Safety
- Energy
- Distance
- Time

Then it can select a suitable action.

Mars Rover Architecture

MARS ENVIRONMENT
↓
SENSORS
↓
PERCEPTS
↓
INTERNAL MODEL
↓
DECISION MAKING
↓
UTILITY EVALUATION
↓
ACTION
↓

ACTUATORS
↓
MARS ENVIRONMENT

Conclusion

The Mars Rover is an example of an intelligent autonomous agent operating in a **partially observable, uncertain, sequential, dynamic and continuous environment**. A model-based utility-based architecture is appropriate because the rover needs to maintain information about its environment and select actions based on safety, scientific value, energy and mission objectives.

3. 8-Queens Problem – Search Space and Problem Formulation

Introduction

The **8-Queens problem** is a classical AI search problem.

The objective is to place **8 queens on an 8 × 8 chessboard** such that no queen attacks another queen.

A chess queen can attack another queen if they are:

- In the same row
- In the same column
- In the same diagonal

Objective

Place eight queens so that:

$$\boxed{\text{No two queens attack each other}}$$

Problem Components

1. Initial State

The initial state is an empty chessboard.

. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .

2. State

A state represents the position of queens placed on the board.

For example:

[2, 4, 1] means:

- Column 1 → Row 2 • Column 2 → Row 4
- Column 3 → Row 1

3. Actions

An action consists of placing a queen in a safe position. For
example:
Place Queen 1
↓
Check safe positions
↓
Place Queen 2
↓
Check safe positions
↓
Continue

4. Transition Model
The transition model describes how the board changes after an action. Example:
Empty board
↓
Place Q1
↓
Place Q2
↓
Place Q3
↓ ...
↓
Place Q8
Each transition adds one queen.

5. Goal Test
A state is a goal state if:

- Eight queens have been placed.
- No two queens are in the same row.
- No two queens are in the same column.
- No two queens are on the same diagonal.

6. Path Cost
Each queen placement can have a cost of 1.
Therefore:
\[8\times1=8 \]
The path cost is usually not the main concern in the 8-Queens problem; finding a valid configuration is the main objective.

Search Space
The search space contains all possible configurations that can be generated while placing queens.
A simple search process is:
Empty Board
↓

Place Queen 1
↓
Possible positions
↓
Place Queen 2
↓
Possible positions
↓
Place Queen 3
↓ ...
↓
Place Queen 8
↓
Goal Test
If a particular path cannot lead to a solution, the algorithm can **backtrack**.

Backtracking
Backtracking is one of the common methods for solving the 8-Queens problem.
Basic idea

1. Place a queen.
2. Check whether it is safe.
3. If safe, place the next queen.
4. If no safe position is available, go back.
5. Move the previous queen to another position.
6. Continue until all 8 queens are placed.

Example Representation One
possible solution is:
[1, 5, 8, 6, 3, 7, 2, 4] This
means:

Column	**Row**
1	1
2	5
3	8
4	6
5	3
6	7
7	2
8	4

The queens are arranged so that none attacks another.

Search Strategies

The problem can be solved using:
1. Breadth First Search Explores
states level by level.
2. Depth First Search
Explores one branch deeply before trying another.
3. Backtracking
Places queens one at a time and returns when a conflict occurs.
4. Hill Climbing
Starts with a configuration and tries to reduce conflicts.
5. Genetic Algorithm
Uses a population of configurations and improves them through selection and mutation.

Conclusion

The 8-Queens problem demonstrates how AI can use **state-space representation and search strategies** to find a solution. Backtracking is particularly suitable because invalid configurations can be rejected early.

4. OLA Cab Booking – Problem-Solving Agent

Problem Statement

A customer wants to travel from one location to another using a cab-booking mobile application such as OLA. The customer enters:

- Pickup location
- Destination

The application displays different cab types such as:

- Micro
- Mini
- Sedan
- Shared
- Prime

The customer chooses a cab based on:

- Fare
- Comfort
- Availability
- Travel time
- Distance
- Personal preference

Suitable Agent A:
\[\boxed{\text{Utility-Based Agent}} \] is
suitable.

Why Utility-Based Agent?

A utility-based agent evaluates different alternatives and selects an action based on its utility. For example:

Cab	Fare	Comfort	Time	Availability

Micro	Low	Medium	Medium	High
Mini	Medium	Medium	Medium	High
Sedan	High	High	Medium	Medium
Shared	Low	Low	High	High
Prime	Very High	Very High	Low	Medium

The customer can give importance to different factors.
For example:
Low cost → Shared/Micro
More comfort → Sedan/Prime

Percepts
The agent receives:

- Pickup location
- Destination
- Available cab types
- Cab locations
- Estimated fare
- Estimated travel time
- Traffic information
- Customer preference
- Driver availability

Actions
The agent can:

1. Search for cabs.
2. Display available cabs.
3. Calculate fare.
4. Estimate travel time.
5. Recommend a cab.
6. Book a cab.
7. Assign driver.
8. Track cab.
9. Cancel booking.
10. Complete the trip.

Goal
The main goal is:
\[\boxed{\text{Provide a suitable cab and transport the customer to the destination.}} \]

Pseudocode
START

Input pickup location
Input destination

```
Calculate distance
Calculate estimated travel time

Search for available cabs

IF no cab is available THEN
    Display "No cabs available"
    STOP
END IF

FOR each available cab type DO

    Get cab type
    Get estimated fare
    Get estimated travel time
    Check availability
    Check comfort level

    Calculate utility value

END FOR

Select suitable cab based on
customer preference and utility

Display cab details

Ask customer for confirmation

IF customer confirms THEN

    Book cab
    Assign driver
    Display driver details
    Track cab

    WHILE destination not reached DO
        Update cab location
        Update estimated arrival time
    END WHILE

    Complete trip
    Calculate final fare
    Generate receipt

ELSE
```

Cancel booking

END IF

STOP

Simple Flowchart
START
↓
Enter Pickup & Destination
↓
Search Available Cabs
↓
Are Cabs Available?
↓ Yes
Calculate Fare, Time & Comfort
↓
Compare Cab Types
↓
Select Suitable Cab
↓
Customer Confirmation
↓
Book Cab
↓
Assign Driver
↓
Track Cab
↓
Reach Destination
↓
Generate Bill
↓
STOP

Conclusion
The OLA cab-booking system can use a **utility-based problem-solving agent** because it has multiple possible choices and must select a suitable cab based on factors such as **cost, comfort, availability, distance, time and customer preferences**.

5. Uniform Cost Search – Logistics Delivery Network

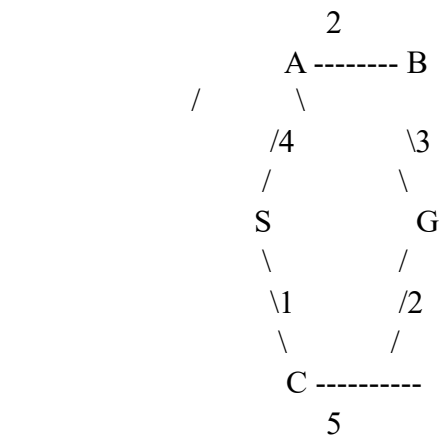
Problem Statement
A logistics company has several warehouses connected through different transportation routes. Each route has a particular cost based on:

- Fuel
- Travel time
- Distance
- Transportation expenses

The company wants to find the **least-cost route** from:
Starting warehouse S → Destination warehouse G Uniform
Cost Search can be used to solve this problem.

What is Uniform Cost Search?
Uniform Cost Search (UCS) is an uninformed search algorithm that expands the node with the **lowest cumulative path cost**.
The cumulative cost is represented by:
\[g(n) \] where:
\[g(n)=\text{cost from starting node S to node n} \] UCS
always chooses:
\[\boxed{\text{Minimum }g(n)} \]

Example Logistics Network
Consider the following warehouse network:



Routes:

From	To	Cost
S	A	4
S	C	1
A	B	2
B	G	3
C	G	2
C	B	5

Possible Paths
Path 1
\[S\rightarrow A\rightarrow B\rightarrow G \]
Cost: \[
4+2+3=9 \]
Therefore:
\[Cost=9 \]

Path 2
\[S\rightarrow C\rightarrow G \]
Cost: \[1+2=3 \] Therefore:
\[Cost=3 \]

Path 3
\[S\rightarrow C\rightarrow B\rightarrow G \]
Cost: \[1+5+3=9 \] Therefore:
\[Cost=9 \]

UCS Execution Step
1 – Start Starting
node:
\[S \]
Path cost:
\[g(S)=0 \] Priority
Queue:
S(0)

Step 2 – Expand S From
S:
\[S\rightarrow A=4 \]\[S\rightarrow C=1 \] Priority
queue becomes:
C(1)
A(4)
UCS selects **C**, because C has the lowest cost.

Step 3 – Expand C Current
cost:
\[g(C)=1 \] From C:
\[C\rightarrow G=2 \]
Total: \[1+2=3 \]
Also: \[C\rightarrow
B=5 \] Total: \[1+5=6
\] Queue:
G(3)
A(4)
B(6)

Step 4 – Select G The
lowest cost is:
\[G(3) \]
Since G is the goal, UCS stops.
Therefore:
\[\boxed{S\rightarrow C\rightarrow G} \]

Final Result
Least-Cost Path
$$\boxed{S\rightarrow C\rightarrow G}$$
Total Cost
$$\boxed{1+2=3}$$
Therefore, the logistics company should use the route:
Starting Warehouse S → Warehouse C → Destination Warehouse G with a total travel cost of **3 units**.

UCS Algorithm
UCS(Start, Goal)

1. Create a priority queue.
2. Insert Start with path cost 0.
3. Create an explored/visited set.

4. While the priority queue is not empty:

    Remove the node with minimum path cost.

    If node is Goal:
        Return the path and cost.

    If node has not been explored:

        Mark node as explored.

        For each neighboring node:

            Calculate new path cost.

            Add the neighbor to
the priority queue.

5. If queue becomes empty:
    Return "No solution".

Important Properties of UCS
1. Complete
UCS is complete when step costs are positive and a solution exists.
2. Optimal
UCS finds the **least-cost solution** when all path costs are non-negative.
3. Evaluation Function UCS uses:
$$\boxed{f(n)=g(n)}$$
Unlike Greedy Best-First Search, it does not use a heuristic.
4. Priority Queue

The frontier is maintained as a priority queue based on cumulative cost.

UCS vs BFS

Feature	BFS	UCS
Selection	Shallowest node	Lowest-cost node
Cost considered	No	Yes
Data structure	Queue	Priority Queue
Optimal	When step costs are equal	For non-negative costs
Main use	Minimum number of steps	Minimum total cost

Overall Short Conclusions

1. Water Jug

The problem is solved using **state-space search**, reaching the goal state (2,0). **2. Mars Rover**

The rover is an intelligent agent operating in a **partially observable and uncertain environment**. A **model-based utility-based architecture** is suitable. **3. 8-Queens**

The problem requires placing eight queens without conflicts. **Backtracking, DFS, and other search strategies** can be used.

4. OLA Cab

A **utility-based agent** is suitable because it compares different cab choices using fare, comfort, time, availability and customer preferences. **5. Logistics**

Uniform Cost Search finds the path having the minimum cumulative cost. In the given example:

$$\boxed{S\rightarrow C\rightarrow G,\quad Cost=3}$$